ELECTRONIC PAYMENT GATEWAY

Complete Pipeline Documentation

MySQL → Kafka → MinIO

Database	MySQL 8.0 (payment_gateway)
Message Broker	Apache Kafka 3.4 (cp-kafka 7.4.1)
Object Storage	MinIO (S3-compatible)
Orchestration	Docker Compose
Language	Python 3.11
Date	March 14, 2026

Table of Contents

1. Architecture Overview

The payment gateway pipeline is a fully containerised data platform built on Docker Compose. It moves data through three distinct stages: generation and storage in MySQL, streaming via Apache Kafka, and archiving in MinIO object storage. The entire stack runs with a single command and requires no external dependencies.

1.1 Pipeline Flow

```
GENERATE (generate_data.py)
   10,000 clients | 1,000 POS | 155,000 transactions | 6 fraud patterns
          |
          v
MYSQL DATABASE (payment_gateway)
   13 tables | InnoDB | utf8mb4 | 14 indexes
          |
          v
STREAM (stream_data.py)
   2 txns/sec | 1 fraud campaign/hr | 2 clients/hr | 1 POS/day
          |
          v
KAFKA PRODUCER (kafka_producer.py)
   Full load on startup + incremental CDC poll every 15s
          |
          v
KAFKA BROKER  (13 topics: pg.transactions, pg.clients, ...)
          |
          v
KAFKA CONSUMER (kafka_consumer.py)
   Buffer 1,000 rows or 30s — write columnar Parquet via PyArrow
          |
          v
MINIO  (s3://payment-gateway/raw/{table}/year=/month=/day/)
   Parquet (Snappy) | Hive-partitioned | native types | self-describing schema
```

1.2 Docker Services

Service	Container	Port	Role
MySQL 8.0	payment_gateway_db	3306	Primary transactional database
Adminer	payment_adminer	8081	Web SQL browser
Zookeeper	zookeeper	2181	Kafka metadata coordination
Kafka	kafka	9092	Message broker
Kafdrop	kafdrop	9000	Kafka topic browser UI
MinIO	minio	9001 / 9002	S3-compatible object storage
data_seeder	payment_data_seeder	—	One-shot bulk data generator
data_streamer	payment_data_streamer	—	Continuous live data generator
kafka_producer	kafka_producer	—	MySQL to Kafka extractor

Service	Container	Port	Role
kafka_consumer	kafka_consumer	—	Kafka to MinIO writer

2. MySQL Database

2.1 Database Setup

The database is created automatically when the MySQL container first starts. Docker mounts the init-scripts/ directory to /docker-entrypoint-initdb.d/ — MySQL executes every .sql file found there in alphabetical order before accepting client connections.

```
# Start MySQL (runs init.sql automatically)
docker-compose up -d mysql

# Connect directly
docker exec -it payment_gateway_db \
  mysql -u pg_user -ppg_password_2024 payment_gateway
```

2.2 Schema — 13 Tables

Table	Category	Rows (seeded)	Primary Key	Description
countries	Reference	10	SMALLINT AI	ISO country codes + default currencies
transaction_types	Reference	7	TINYINT AI	PURCHASE, REFUND, WITHDRAWAL, etc.
transaction_statuses	Reference	7	TINYINT AI	APPROVED, DECLINED, SETTLED, etc.
card_networks	Reference	4	TINYINT AI	Visa, Mastercard, Amex, Discover
merchant_categories	Reference	15	SMALLINT (MCC)	ISO Merchant Category Codes
merchants	Merchant	200	INT AI	Registered businesses
pos_terminals	Merchant	1,000+	INT AI	Physical/virtual payment devices
clients	Client	10,000+	INT AI	Registered cardholders
payment_cards	Client	15,000-25,000	INT AI	Tokenised PAN instruments
transactions	Transaction	150,000+	BIGINT AI	Core financial events
fraud_alerts	Risk	Varies	BIGINT AI	Fraud rules triggered per transaction
refunds	Risk	~5% of txns	BIGINT AI	Refund and chargeback records
settlement_batches	Settlement	73,000+	INT AI	Daily merchant payout aggregates

2.3 Key Table: transactions

The transactions table is the core of the schema — every payment event regardless of channel or status lands here as a single immutable row.

Column	Type	Description
txn_id	BIGINT UNSIGNED AI	Surrogate primary key
reference	CHAR(36) UNIQUE	UUID v4 external idempotency key
client_id	INT UNSIGNED FK	Initiating cardholder
card_id	INT UNSIGNED FK	Card used (NULL for ATM cash)
pos_id	INT UNSIGNED FK	Terminal (NULL for online)
merchant_id	INT UNSIGNED FK	Destination merchant
amount	DECIMAL(14,2)	Gross transaction amount
currency	CHAR(3)	ISO 4217 currency code
fee	DECIMAL(10,2)	Gateway processing fee
net_amount	DECIMAL(14,2) GENERATED	amount - fee (stored, computed by MySQL)
channel	ENUM	POS | ONLINE | ATM | MOBILE | IVR
is_flagged	TINYINT(1)	1 = fraud detection rule triggered
initiated_at	DATETIME	When client submitted the transaction

2.4 Indexes

14 secondary indexes optimise the most frequent query patterns against the transactions table:

Index	Column	Purpose
idx_txn_client	client_id	All transactions for a given client
idx_txn_merchant	merchant_id	All transactions for a merchant
idx_txn_initiated	initiated_at	Date/time range queries and daily volume reports
idx_txn_flagged	is_flagged	Instant retrieval of all fraud-flagged rows
idx_txn_status	status_id	Filter by APPROVED / DECLINED / SETTLED
idx_txn_channel	channel	Channel breakdown analytics
idx_fraud_txn	txn_id (fraud_alerts)	Join from transactions to alerts
idx_settlement_date	batch_date	Date-range queries on settlement history

3. Data Generation (generate_data.py)

The bulk seeder runs once on first deployment and populates the entire historical dataset. It includes an idempotency guard — if the transactions table already contains 1,000 or more rows the script exits immediately with code 0, preventing duplicate seeding.

> *Idempotency guard: if COUNT(*) FROM transactions >= 1000 → exit 0 without inserting anything. Safe to restart.*

3.1 What Gets Generated

Entity	Volume	Key Details
Merchants	200	95% active, spread across all countries and MCC codes
POS Terminals	1,000	97% active, 4-5 per merchant on average
Clients	10,000	96% active, registered 1-2 years before seed date
Payment Cards	15,000 - 25,000	1-4 per client, one marked is_default=1
Transactions	155,000+	80% normal, 20% fraud across 6 patterns
Fraud Alerts	~31,000+	1-3 rules per flagged transaction, 70% reviewed
Refunds	~5% of approved	Mix of full and partial; 30% partial
Settlement Batches	73,000	200 merchants x 365 days

3.2 Transaction Distribution

Parameter	Value / Distribution
Amount	Log-normal: most are small ($1-$200), rare large ($1,000-$4,000)
Status	73% APPROVED, 14% DECLINED, 8% SETTLED, 3% PENDING, 2% FAILED
Channel	55% POS, 25% ONLINE, 10% MOBILE, 8% ATM, 2% IVR
Currency	50% USD, 15% EUR, 10% GBP, 8% EGP, remainder split across 5 currencies
Period	Uniformly distributed over the previous 365 days
Fraud rate	20% of all transactions carry is_flagged=1

3.3 Fraud Patterns

Six distinct fraud archetypes are injected into the dataset, each simulating a real-world attack vector:

Pattern	Transactions	Key Signals
Velocity Burst	5-20 per campaign	Same client, shared IP/device, 10-60s between transactions

Pattern	Transactions	Key Signals
Card Testing	10-30 per campaign	Micro-amounts $0.01-$1.99, ONLINE channel, 50% decline rate
High-Value Fraud	1 per campaign	Amount $3k-$10k, foreign currency, ONLINE or MOBILE only
Night Owl	2-8 per campaign	Timestamps between 01:00-04:59, ONLINE/MOBILE channel
International Spree	3-10 per campaign	Multiple foreign currencies, is_international=1, rapid succession
Account Takeover	1 per campaign	New device_id, foreign IP, high amount

3.4 Running the Seeder

```
# Default: 10,000 clients, 1,000 POS, 155,000 transactions
python generate_data.py

# Custom volume
python generate_data.py \
  --host localhost --port 3306 \
  --user pg_user --password pg_password_2024 \
  --database payment_gateway \
  --clients 10000 --pos 1000 --transactions 155000

# In Docker — runs automatically on first boot
docker logs -f payment_data_seeder
```

4. Real-Time Data Streaming (stream_data.py)

After the seeder completes, the streamer runs indefinitely at precisely controlled rates. It uses PrecisionTicker threads that self-correct for clock drift so the actual throughput matches the configured rate exactly over time.

4.1 Stream Rates

Event	Rate	Interval	Description
Normal Transaction	2 / second	Every 0.5s	Card transactions across all channels and statuses
Fraud Campaign	1 / hour	Every 3,600s	One random pattern, burst of flagged transactions + alerts
New Client	2 / hour	Every 1,800s	New cardholder + 1-3 cards, added to live cache immediately
New POS Terminal	1 / day	Every 86,400s	New terminal for random merchant, added to live cache

4.2 How PrecisionTicker Works

Each event type runs in its own daemon thread. The ticker measures real elapsed time and adjusts the next sleep interval to compensate for any drift caused by database write latency:

```
class PrecisionTicker(threading.Thread):
    def run(self):
        next_tick = time.perf_counter()
        while RUNNING.is_set():
            if time.perf_counter() >= next_tick:
                threading.Thread(target=self.callback).start()
                next_tick += self.interval   # advance by exact interval
            # sleep for remaining time minus 1ms buffer
            time.sleep(max(0, next_tick - time.perf_counter() - 0.001))
```

4.3 Fraud Campaign Streaming

Every hour the fraud ticker fires and randomly selects one of six patterns. All transactions in a campaign share realistic signals — the same IP and device fingerprint for velocity bursts, the same client for card testing, consistent foreign currencies for international sprees:

```
# Example live output
🚨  Fraud campaign fired | 🌍 International Spree     |  7 txns
🚨  Fraud campaign fired | 💨 Velocity Burst           | 14 txns
🚨  Fraud campaign fired | 🔍 Card Testing             | 23 txns
```

4.4 Live Status Dashboard

The streamer prints a live dashboard line every 5 seconds showing real-time throughput:

```
⏱  3605s | 💰   7213 txns (2.00/s) | 👤  2 clients (2.00/hr) | 🖥️  0 POS | 🚨  1
campaigns (7 fraud txns, 1.00/hr) | ❌ 0 err
```

4.5 Running the Streamer

```
# Default rates
python stream_data.py

# Custom rates
python stream_data.py \
  --txn-per-second 5 \
  --clients-per-hour 10 \
  --fraud-per-hour 6 \
  --pos-per-day 2

# In Docker — starts automatically after seeder finishes
docker logs -f payment_data_streamer
```

5. Kafka Producer (kafka_producer.py)

The producer extracts data from MySQL and publishes it to Kafka topics. It operates in two sequential phases: a one-time full load of all historical data, followed by a continuous incremental CDC poll that captures new rows as they are written by the streamer.

5.1 Two-Phase Operation

Phase	Trigger	What Happens
Phase 1 — Full Load	Once on startup	Reads every row from all 13 tables in batches of 500. Skipped if the topic already contains data (idempotent restart).
Phase 2 — CDC Poll	Every 15 seconds forever	Queries each table for rows with created_at newer than the last watermark. Publishes only the delta. Static reference tables are skipped.

5.2 Kafka Topics

One topic is created per table, all prefixed with "pg.":

Topic	Source Table	Partitions	Retention
pg.transactions	transactions	3	7 days
pg.clients	clients	3	7 days
pg.fraud_alerts	fraud_alerts	3	7 days
pg.merchants	merchants	3	7 days
pg.pos_terminals	pos_terminals	3	7 days
pg.payment_cards	payment_cards	3	7 days
pg.refunds	refunds	3	7 days
pg.settlement_batches	settlement_batches	3	7 days

5.3 Message Format

Every Kafka message is a JSON object with a metadata envelope and a payload containing the row:

```
{
  "_meta": {
    "table":  "transactions",
    "op":     "r",        // r=full-load read, c=CDC new row
    "ts_ms":  1712345678000
  },
  "payload": {
    "txn_id":       1042,
    "reference":    "a3f2c1d0-...",
    "client_id":    504,
    "amount":       "142.50",
    "currency":     "USD",
```

```
    "channel":      "ONLINE",
    "is_flagged":   0,
    "initiated_at": "2025-03-14T13:45:22"
  }
}
```

5.4 Watermark-Based CDC

The producer tracks a high-watermark timestamp per table. On each poll cycle it runs a query like the following and advances the watermark to the latest created_at seen:

```
SELECT * FROM transactions
WHERE  created_at > '2025-03-14 13:45:00'   -- last watermark
ORDER  BY txn_id;
```

5.5 Producer Configuration

Parameter	Value	Purpose
acks	all	Wait for all in-sync replicas — no data loss
retries	5	Retry transient send failures
batch_size	65536 (64 KB)	Coalesce small messages for efficiency
linger_ms	20	Wait up to 20ms to fill a batch
compression_type	gzip	Reduce network bandwidth
max_request_size	10 MB	Allow large batch sends

6. Kafka Consumer (kafka_consumer.py)

The consumer subscribes to all pg.* topics, buffers messages per table, and writes columnar Apache Parquet files to MinIO. Parquet is a binary columnar format — it compresses 3-10x better than JSON, is directly queryable by Pandas, PySpark, DuckDB, Trino, and Athena without any ETL, and preserves native MySQL data types across the pipeline.

6.1 Why Parquet Instead of JSON

Dimension	JSON-Lines (.jsonl.gz)	Parquet (.parquet)
Storage size	Larger — text encoding of every value	3-10x smaller — columnar + dictionary encoding
Query performance	Full row scan required	Column pruning — only reads needed columns
Data types	All values become strings	Native int, float, timestamp, decimal preserved
Compatibility	Universal but slow for analytics	Pandas, Spark, DuckDB, Trino, Athena, BigQuery native
Schema	Inferred per row	Embedded in file footer — self-describing
Compression	gzip on whole file	Per-column Snappy/gzip/zstd — better ratios

6.2 MinIO Object Layout

Files use Hive-style date partitioning so downstream tools (Spark, Trino, Athena) can apply partition pruning — they only read partitions matching the query date range:

```
s3://payment-gateway/
  raw/
    transactions/
      year=2025/month=03/day=14/
        20250314_153022_abc123.parquet    <- 1,000 rows, ~180 KB (Snappy)
        20250314_153552_def456.parquet
    clients/
      year=2025/month=03/day=14/
        20250314_153500_ghi789.parquet
    fraud_alerts/  merchants/  pos_terminals/
    payment_cards/ refunds/    settlement_batches/
```

6.3 How Parquet is Written

The consumer buffers raw Python dicts in memory instead of JSON strings. When a flush is triggered, PyArrow converts the entire batch to a columnar Parquet file in one pass:

```
# Step 1 — buffer stores dicts (not strings)
buffers["transactions"].append({
    "txn_id": 1042, "amount": 142.50,
    "currency": "USD", "is_flagged": 0, ...
})

# Step 2 — flush pivots rows to columns and writes Parquet
def rows_to_parquet(rows):
```

```
    rows  = [normalise_row(r) for r in rows]   # Decimal->float, etc.
    cols  = {k: [r.get(k) for r in rows]       # pivot row->column
             for k in rows[0]}
    table = pa.Table.from_pydict(cols)          # schema inferred by PyArrow
    pq.write_table(table, buf,
        compression="snappy",
        write_statistics=True,   # min/max for predicate pushdown
        use_dictionary=True,     # dict encoding for low-cardinality cols
    )

# Step 3 — upload to MinIO
client.put_object(bucket, key, data=buf, length=size)
```

6.4 Type Normalisation

MySQL types need normalisation before PyArrow can infer a consistent schema:

MySQL Type	Python Type	Stored in Parquet As
DECIMAL(14,2)	decimal.Decimal	float64 (converted)
DATETIME	datetime.datetime	timestamp[us]
DATE	datetime.date	date32
TINYINT(1)	int	int8
VARCHAR / CHAR	str	utf8 string
NULL	None	null in column array

6.5 Flush Strategy

Condition	Value	Rationale
Row threshold	1,000 rows	Prevents memory growth; produces right-sized Parquet files
Time threshold	30 seconds	Guarantees files land in MinIO every 30s regardless of volume
Background timer	Checks every 5s	Catches low-traffic tables that rarely hit the row threshold
Shutdown flush	Immediate on SIGTERM	All in-memory buffers flushed — no data loss on graceful stop

6.6 Compression Options

The compression codec is configurable via the PARQUET_COMPRESSION environment variable in docker-compose.yml:

Codec	Speed	Ratio	Best For
snappy (default)	Fastest	Good (~3x)	Interactive analytics — low read latency
gzip	Slower	Better (~5x)	Cold storage — maximum space saving
zstd	Fast	Best (~6x)	Balanced — recommended for production

Codec	Speed	Ratio	Best For
none	Fastest	None (1x)	Debugging only

```
# docker-compose.yml — kafka_consumer environment
PARQUET_COMPRESSION: zstd
```

6.7 Consumer Configuration

Parameter	Default	Purpose
group_id	pg-minio-consumer	Consumer group — Kafka tracks offset per group
auto_offset_reset	earliest	On first run, start from the very first message
max_poll_records	500	Rows returned per Kafka poll call
fetch_max_bytes	50 MB	Maximum bytes fetched per broker request
FLUSH_ROWS	1,000	Row count threshold to trigger a Parquet write
FLUSH_SECONDS	30	Time threshold to trigger a Parquet write
PARQUET_COMPRESSION	snappy	Column compression codec

6.8 Reading the Parquet Files

Files written by the consumer are self-describing and readable by any analytics tool without transformation:

```
# Pandas
import pandas as pd
df = pd.read_parquet(
    "s3://payment-gateway/raw/transactions/year=2025/month=03/day=14/",
    storage_options={"endpoint_url": "http://localhost:9002",
                     "key": "minioadmin", "secret": "minioadmin"}
)
print(df.dtypes)   # int64, float64, datetime64, object — native types

# DuckDB (query without loading into memory)
import duckdb
duckdb.sql("""
    SELECT channel, COUNT(*) AS txns,
           SUM(amount) AS revenue,
           AVG(is_flagged) AS fraud_rate
    FROM   read_parquet('s3://payment-gateway/raw/transactions/**/*.parquet')
    GROUP  BY channel ORDER BY revenue DESC
""").show()

# PySpark
df = spark.read.parquet(
    "s3a://payment-gateway/raw/transactions/"
).filter("year=2025 AND month=3")   # partition pruning
```

6.9 Monitoring Consumer Lag

```
# Kafdrop UI
http://localhost:9000

# CLI — how far behind is the consumer?
docker exec -it kafka \
  kafka-consumer-groups \
  --bootstrap-server localhost:9092 \
  --describe --group pg-minio-consumer

# Consumer stats (logged every 30s)
docker logs -f kafka_consumer | grep "Stats"
```

7. Docker & Infrastructure

7.1 Boot Sequence

Docker Compose enforces a strict startup order through healthchecks and depends_on conditions:

```
Zookeeper  →  healthy (cub zk-ready → "imok")
    |
    v
Kafka      →  healthy (cub kafka-ready → broker metadata reachable)
    |
    v
MySQL      →  healthy (mysqladmin ping OK) → init.sql runs
    |
    v
data_seeder  →  seeds 155,000 rows → exits 0
    |
    v
data_streamer  →  2 txns/s, 1 fraud/hr  (runs forever)
kafka_producer →  full load + CDC poll  (runs forever)
    |
    v
MinIO      →  healthy (curl /minio/health/live → 200)
    |
    v
kafka_consumer →  subscribes to pg.* → writes MinIO  (runs forever)
```

7.2 Healthchecks

Service	Tool	Command	What It Tests
Zookeeper	cub	cub zk-ready localhost:2181 30	ZooKeeper 4LW ruok/imok protocol
Kafka	cub	cub kafka-ready -b localhost:9092 1 40	Full broker metadata round-trip
MySQL	mysqladmin	mysqladmin ping -u pg_user	Database accepts connections
MinIO	curl	curl /minio/health/live	MinIO HTTP health endpoint returns 200

7.3 Pipeline Entrypoint

The pipeline containers (kafka_producer and kafka_consumer) use a pure Python entrypoint instead of a shell script. This eliminates all shell compatibility issues (bash vs dash, /dev/tcp availability, heredoc stdin consumption):

```
# entrypoint_pipeline.py — key logic
def wait_kafka():
    host = os.environ.get("KAFKA_HOST", "kafka")
    port = int(os.environ.get("KAFKA_PORT", "9092"))
    # socket.create_connection — pure stdlib, works everywhere
    for i in range(1, MAX_RETRIES + 1):
        try:
            with socket.create_connection((host, port), timeout=3):
                return   # success
```

```
        except OSError:
            time.sleep(INTERVAL)

# After all services ready — os.execvp replaces this process
os.execvp(cmd[0], cmd)   # e.g. ["python3", "kafka_producer.py"]
```

7.4 Common Commands

Action	Command
Start everything	docker-compose up -d
Rebuild after code changes	docker-compose up --build
Full reset (wipe volumes)	docker-compose down -v && docker-compose up --build
Watch seeder progress	docker logs -f payment_data_seeder
Watch streamer live	docker logs -f payment_data_streamer
Watch producer	docker logs -f kafka_producer
Watch consumer	docker logs -f kafka_consumer
MySQL shell	docker exec -it payment_gateway_db mysql -u pg_user -ppg_password_2024 payment_gateway
Check consumer lag	docker exec -it kafka kafka-consumer-groups --bootstrap-server localhost:9092 --describe --group pg-minio-consumer

8. Useful SQL Queries

8.1 Transaction Volume Per Day

```
SELECT DATE(initiated_at) AS day,
       COUNT(*)            AS txn_count,
       SUM(amount)         AS gross,
       SUM(is_flagged)     AS fraud_count,
       ROUND(SUM(is_flagged)/COUNT(*)*100,2) AS fraud_pct
FROM   transactions
GROUP  BY day
ORDER  BY day DESC;
```

8.2 Top 10 Merchants by Revenue

```
SELECT m.trade_name,
       COUNT(t.txn_id) AS txns,
       SUM(t.amount)   AS revenue
FROM   transactions t
JOIN   merchants m ON t.merchant_id = m.merchant_id
JOIN   transaction_statuses s ON t.status_id = s.status_id
WHERE  s.code = 'APPROVED'
GROUP  BY m.merchant_id
ORDER  BY revenue DESC LIMIT 10;
```

8.3 Open Fraud Alerts

```
SELECT fa.alert_id, fa.rule_triggered, fa.risk_score,
       t.amount, t.channel, t.initiated_at,
       CONCAT(c.first_name,' ',c.last_name) AS client
FROM   fraud_alerts fa
JOIN   transactions t ON fa.txn_id   = t.txn_id
JOIN   clients c      ON t.client_id = c.client_id
WHERE  fa.is_confirmed IS NULL
ORDER  BY fa.risk_score DESC, t.amount DESC;
```

8.4 Fraud Rate by Channel

```
SELECT channel,
       COUNT(*) AS total,
       SUM(is_flagged) AS flagged,
       ROUND(SUM(is_flagged)/COUNT(*)*100,2) AS fraud_pct
FROM   transactions
GROUP  BY channel
ORDER  BY fraud_pct DESC;
```

8.5 Settlement Summary (Last 30 Days)

```
SELECT m.trade_name,
       SUM(sb.gross_amount) AS gross,
```

```
       SUM(sb.total_fees)   AS fees,
       SUM(sb.net_payout)   AS net
FROM   settlement_batches sb
JOIN   merchants m ON sb.merchant_id = m.merchant_id
WHERE  sb.batch_date >= CURDATE() - INTERVAL 30 DAY
  AND  sb.status = 'SETTLED'
GROUP  BY m.merchant_id
ORDER  BY net DESC;
```

9. Troubleshooting

Symptom	Root Cause	Fix
generate_data crashes: IndexError on mcc_list	Duplicate MCC 5411 in init.sql caused the merchant_categories INSERT to silently roll back, leaving the table empty.	Fixed: second MCC 5411 changed to 5400. Run: docker-compose down -v && docker-compose up --build
data_seeder cannot connect	MySQL healthcheck passes when TCP is accepted, but init.sql is still running and tables do not exist yet.	Fixed: entrypoint.sh two-phase wait — Phase 1 TCP, Phase 2 polls information_schema for transaction_statuses.
Kafka container unhealthy forever	Previous healthchecks used nc, kafka-topics, or bash /dev/tcp — none reliably available in cp-kafka image.	Fixed: use "cub kafka-ready" which ships in every Confluent image and does a real broker metadata probe.
[Kafka] Attempt N/90 from pipeline container	Multiple causes across iterations: bash not installed, heredoc stdin consumption, host.docker.internal unresolvable on Linux.	Fixed: replaced shell entrypoint with pure Python entrypoint_pipeline.py using socket.create_connection.
Zookeeper logs Unable to read additional data	Healthcheck opened a raw TCP connection without sending the ZK protocol. ZK logged it as an unexpected disconnect.	Fixed: cub zk-ready sends proper ZK protocol. Added ZOOKEEPER_4LW_COMMANDS_WHITELIST to enable ruok.
Adminer/Airflow port 8080 conflict	Both services mapped to host port 8080.	Fixed: adminer on 8081, airflow-webserver on 8082.
MinIO data lost on restart	No volume declared for MinIO /data directory.	Fixed: minio_data volume added and mounted to /data.